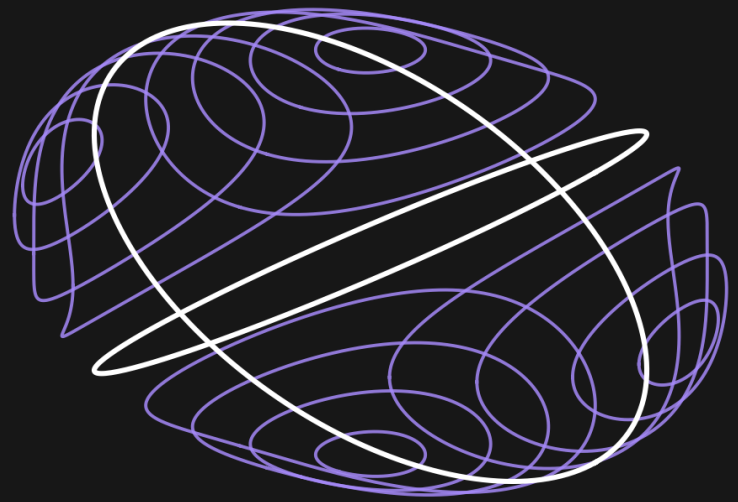
Máster Universitario en Sistemas Espaciales

AMPLIACIÓN DE MATEMÁTICAS 1 · PEI 1

The Intermediate Axis Theorem

Rigid-body dynamics, spectral analysis and a reactive Python tool



Ampliación de Matemáticas 1 · Course 2026–2027

Rubén Núñez Cuenca, Mario Penin Fernández and Ignacio Pérez González

A wing nut flips in orbit with no torque acting on it

Dzhanibekov, Salyut 7, 1985

A wing nut spinning about its intermediate axis reverses every few seconds. Rigid, torque-free: our model.

PENDING · real photo

Explorer 1, 1958

Spin about the minimum axis decays to the maximum axis. Cause: internal energy dissipation, outside a rigid model.

PENDING · real photo

1 · Physics · Rubén Núñez

Model · invariants · linearisation · Lyapunov · the flip



Three coupled equations describe a torque-free rigid body

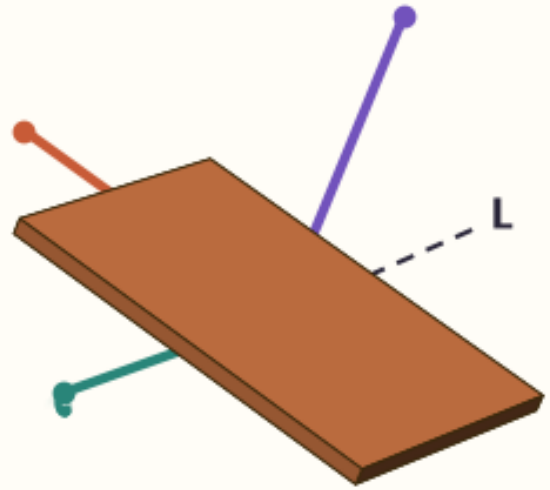
Body frame, principal axes, $I_1 < I_2 < I_3$

$$I_1 \, d\omega_1/dt = (I_2 - I_3) \, \omega_2 \, \omega_3$$

$$I_2 \, d\omega_2/dt = (I_3 - I_1) \, \omega_3 \, \omega_1$$

$$I_3 \, d\omega_3/dt = (I_1 - I_2) \, \omega_1 \, \omega_2$$

State $U = \omega = (\omega_1, \omega_2, \omega_3)$. Attitude added through a quaternion: $dq/dt = ½ \, q \otimes (0, \omega)$.



Source: active/space_physics/eje_intermedio.py · PEI1(), Attitude_equations()

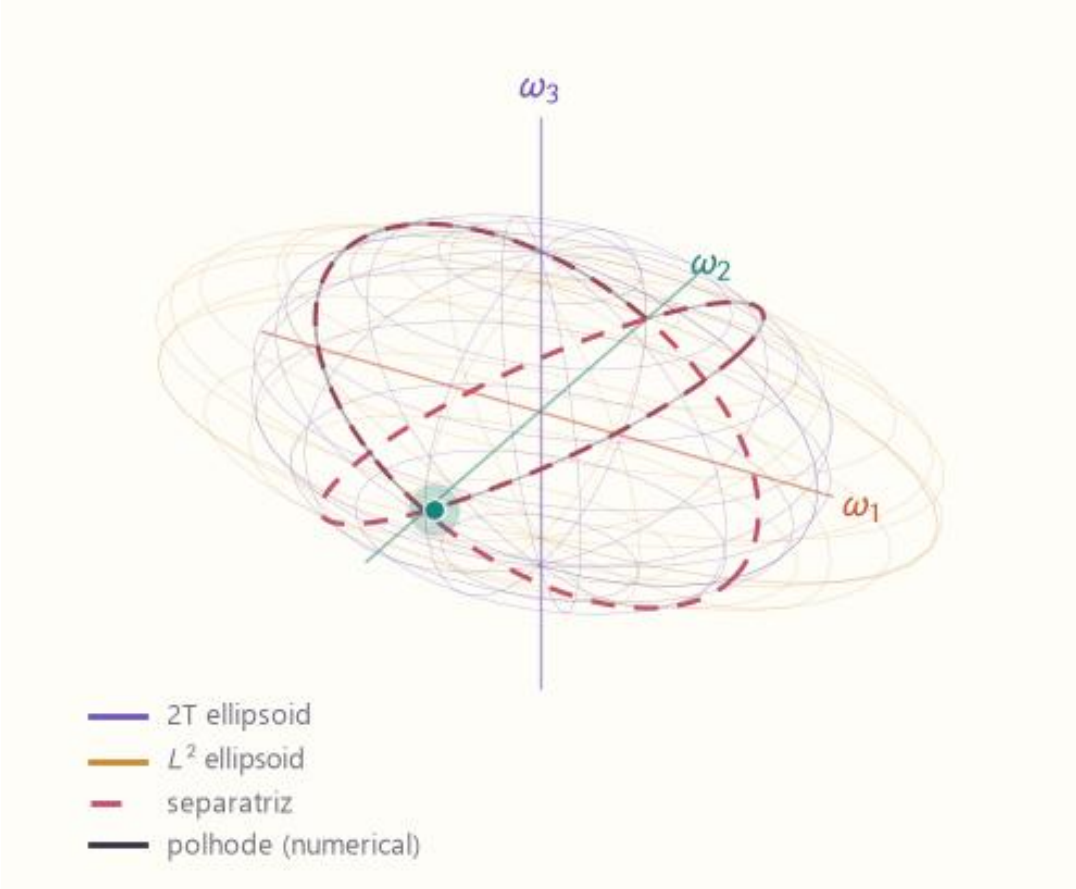
Two invariants confine ω to the intersection of two ellipsoids

$$2T = I_1\omega_1^2 + I_2\omega_2^2 + I_3\omega_3^2$$

$$L^2 = I_1^2\omega_1^2 + I_2^2\omega_2^2 + I_3^2\omega_3^2$$

Multiply equation k by ω_k (or I_k ω_k) and add: the right-hand sides cancel because $(I_2 - I_3) + (I_3 - I_1) + (I_1 - I_2) = 0$.

Motion exists only if $2T \cdot I_1 \le L^2 \le 2T \cdot I_3$. The polhode is the curve where both surfaces meet.



Source: Kinetic_energy_2T(), Angular_momentum_L2(), Energy_ellipsoid(), Momentum_ellipsoid() · GUI page Motion

Linearised at each pure spin, only the intermediate axis is a saddle

Spin axis	Equilibrium	Eigenvalues of J	I = (1, 2, 3), Ω = 1	Type
Minimum I_1	(Ω, 0, 0)	$\pm i\Omega \sqrt{[(I_3-I_1)(I_2-I_1)/(I_2 I_3)]}$	± 0.577 i	centre
Intermediate I_2	**(0, Ω, 0)**	**$\pm \Omega \sqrt{[(I_3-I_2)(I_2-I_1)/(I_1 I_3)]}$**	**± 0.577**	**saddle**
Maximum I_3	(0, 0, Ω)	$\pm i\Omega \sqrt{[(I_3-I_2)(I_3-I_1)/(I_1 I_2)]}$	± 1.000 i	centre

Source: euler_jacobian(), Linear_eigenvalues() · PEI 1 statement, eqs. (9), (10), (12)

A linear centre proves nothing; a conserved Lyapunov function does

Minimum axis

$V = L^2 - I_1 \cdot 2T = I_2(I_2 - I_1)\omega_2^2 + I_3(I_3 - I_1)\omega_3^2$

positive definite: stable

Intermediate axis

$V = L^2 - I_2 \cdot 2T = I_1(I_1 - I_2)\omega_1^2 + I_3(I_3 - I_2)\omega_3^2$

indefinite: V = 0 are the separatrix planes

Maximum axis

$V = L^2 - I_3 \cdot 2T = I_1(I_1 - I_3)\omega_1^2 + I_2(I_2 - I_3)\omega_2^2$

negative definite: stable

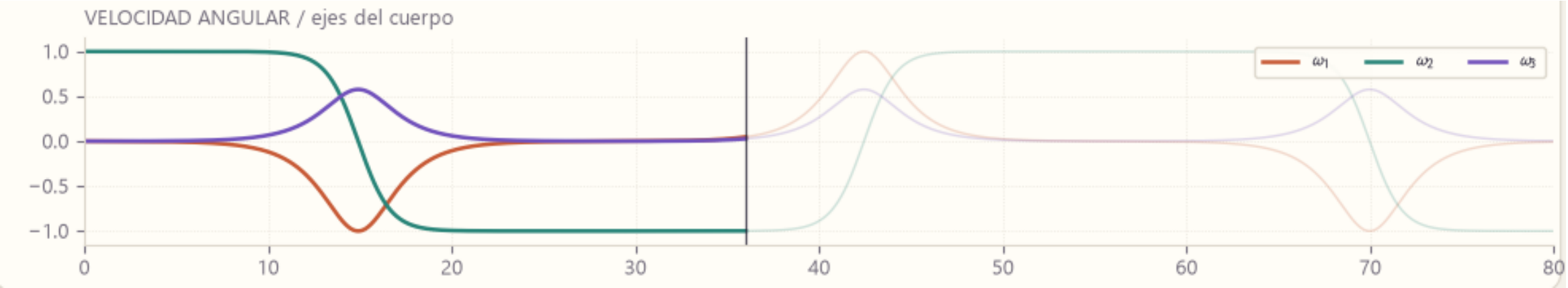
Source: Energy_Casimir() in eje_intermedio.py

The flip follows the separatrix, and its timing is predictable

Separatrix: two planes $\omega_3 = \pm c\,\omega_1$, $c = \sqrt{[I_1(I_2-I_1) / I_3(I_3-I_2)]}$

Period $P = 4K(k)/\nu$, infinite on the separatrix. Exact 55.062 s, measured 55.062 s.

First flip ≈ $(1/\lambda)\ln(1/\varepsilon)$: 14.9 s, 34.9 s and 50.8 s for $\varepsilon = 10^{-3}, 10^{-8}, 10^{-12}$; slope 1.732 s = $1/\lambda$.



ω(t) in body axes, intermediate axis, $\varepsilon = 10^{-3}$

Source: Separatrix(), Polhode_period(), Elliptic_K() · flip times measured with RK4, Δt = 0.02, I = (1, 2, 3)

2 · Numerical engine · Mario Penin

Cauchy problem · Newton · Richardson · measured stability regions



One engine integrates any F(U, t) and never sees the physics

F(U, t)

space_physics: Euler + quaternion, exact Jacobian attached as F.jacobian

→

Scheme

Euler, Crank-Nicolson, RK4: U_{n+1} = method(U_n, Δt, t_n, F)

→

Cauchy_problem

Fixed mesh, or adaptive by step doubling (Richardson on one step)

→

Integration_stopped

Newton failure or non-finite state: returns the partial solution

Source: active/numerical_engine/Cauchy_Problem.py · temporal_schemes.py

An implicit step is a Newton solve, not a fixed-point loop

First version: fixed point

$Y \leftarrow U + \Delta t/2\,[F(U) + F(Y)]$, tolerance 1e-6, no iteration cap. Converges only if $\Delta t \cdot \rho(J)/2 < 1$; for small Δt it returned the Euler predictor (CN identical to Euler at N = 16 000).

Current: Newton, exact Jacobian

Solve $G(X) = X - U - \Delta t/2\,[F(U) + F(X)] = 0$ with $G' = I - \Delta t/2\,J(X)$. Euler predictor, tolerance 1e-10, 25 iterations, failure raised as an exception.

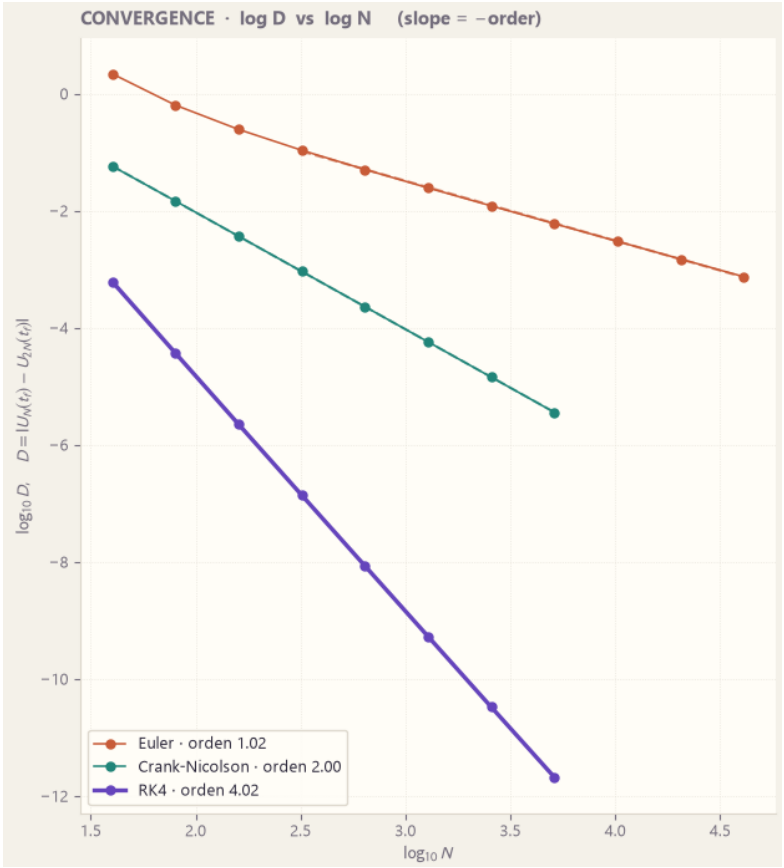
Source: paso_crank_nicolson() in PEI1/PEI1.py · limitations table in numerical_engine/README.md (8 Oct)

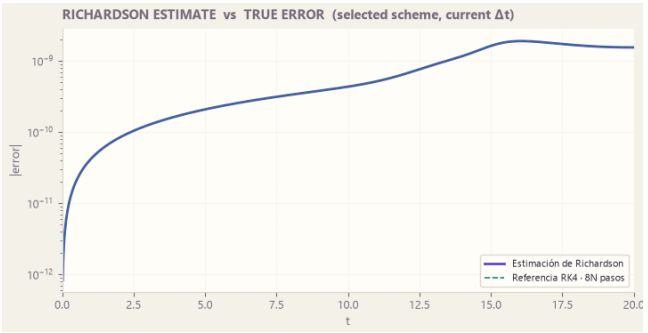
Richardson measures the order on the rigid body itself

D = |U_N(t_f) − U_2N(t_f)| ≈ E_N (1 − 2^−q): no exact solution needed.

Measured order, intermediate axis, t_f = 20: Euler 1.02, Crank-Nicolson 2.00, RK4 4.02.

The fitted range is chosen automatically: the longest run where the local order $\log_2(D_k / D_{k+1})$ stays within 15 %.



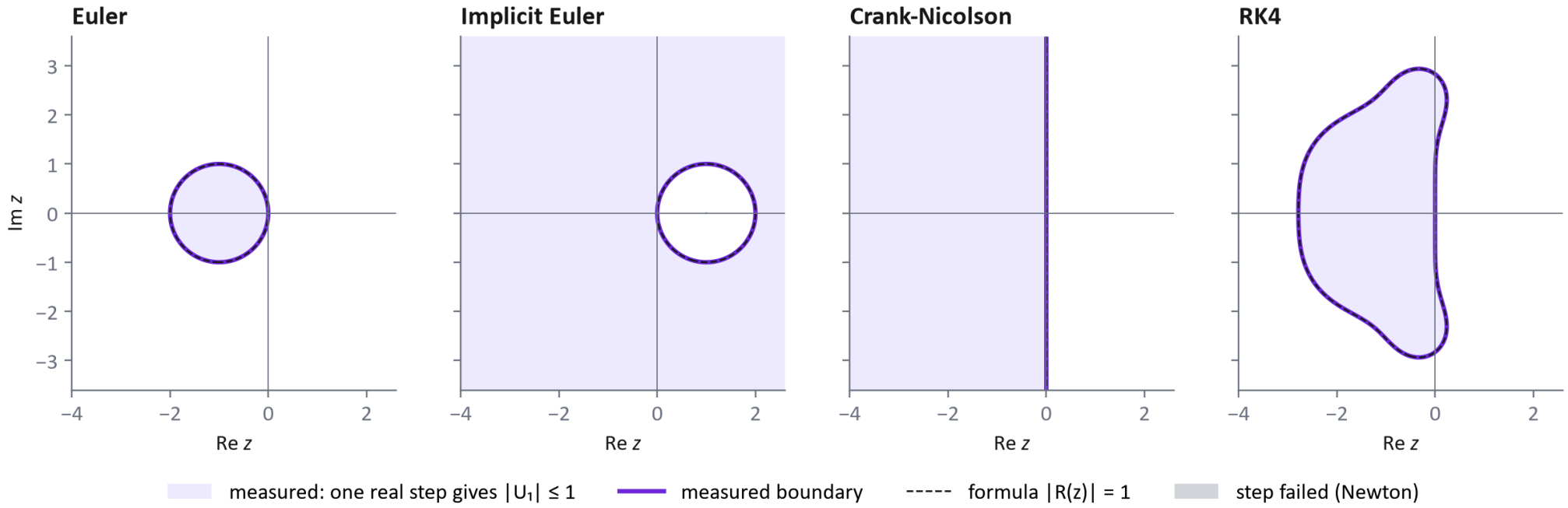


RK4: Richardson estimate (solid) on the reference error (dashed)

Source: curva_convergencia(), tramo_recto(), ordenes_locales() · GUI page Precision

We measured the stability regions by running our own schemes

One step of size 1 on $dU/dt = \lambda U$ from $U_0 = 1$, for every $z = \lambda$ on a 133 × 145 grid. $|U_1|$ is the amplification. Our Newton is real-valued, so $\lambda = a + ib$ runs as the real system $A = [a\ -b;\ b\ a]$.



Largest gap between measured $|U_1|$ and the formula $|R(z)|$: Euler $3 \cdot 10^{-16}$, implicit Euler $2 \cdot 10^{-12}$ (finite-difference Jacobian), Crank-Nicolson $2 \cdot 10^{-14}$, RK4 $1 \cdot 10^{-14}$. One failed step: implicit Euler at $z = 1$, where the system is singular.

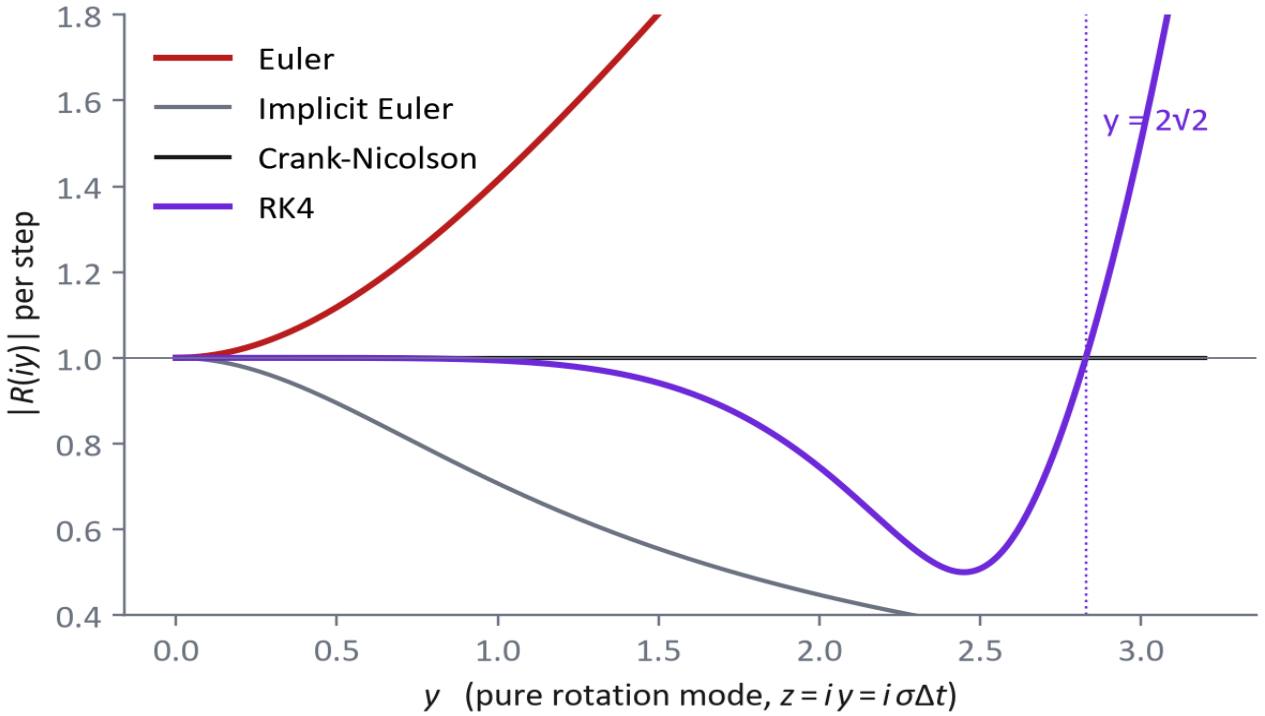
Source: presentacion/figuras/regiones_numericas.py · method of Stability_Region, jahrWork/MUSE_orbits_S1

Rotation modes live on the imaginary axis, where the schemes part ways

$z = \lambda \Delta t$ with $\Delta t = 0.02$: ± 0.0115 (axis 2, real), $\pm 0.0115\,i$ and $\pm 0.02\,i$ (axes 1, 3), $\pm 0.01\,i$ (quaternion).

Euler $|R(iy)| = \sqrt{(1 + y^2)} > 1$: every rotation mode grows. Crank-Nicolson: exactly 1. RK4: below 1 up to $y = 2\sqrt{2}$.

RK4 stays stable on axis 3 for $\Delta t < 2\sqrt{2}/\sigma = 2.83$ s; Euler is never stable there.

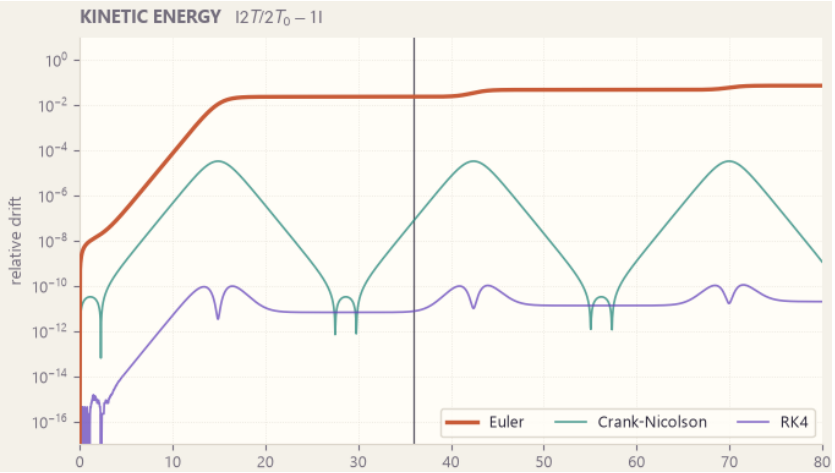


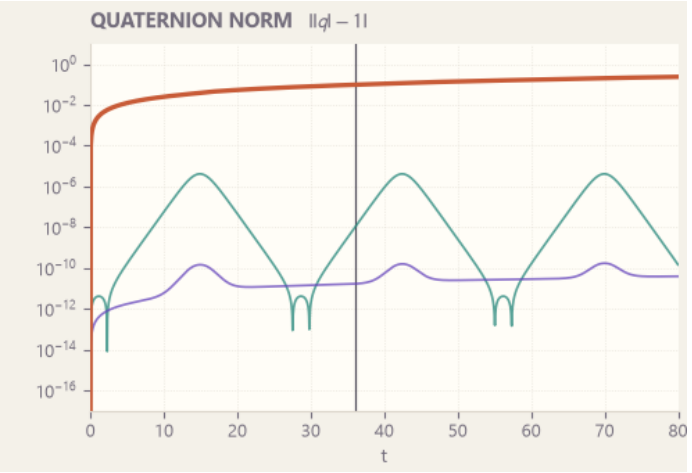
Amplification per step on the imaginary axis

Source: stability_function() in Estabilidad.py · I = (1, 2, 3), Ω = 1, Δt = 0.02

R(z) predicts the drift the simulation shows

Scheme	max drift of 2T and L^2	$\|\|q(t_f)\|\| - 1$ predicted by $\prod R(z_n)$	$\|\|q(t_f)\|\| - 1$ simulated
Euler	**$7.3 \cdot 10^{-2}$**	**$2.405 \cdot 10^{-1}$**	**$2.405 \cdot 10^{-1}$**
Crank-Nicolson	$3.3 \cdot 10^{-5}$	$-1.3 \cdot 10^{-14}$	$-1.5 \cdot 10^{-10}$
RK4	$1.1 \cdot 10^{-10}$	$-3.2 \cdot 10^{-11}$	$-3.9 \cdot 10^{-11}$





2T drift in time: Euler (red), CN (green), RK4 (purple)

|q| drift in time

Source: GUI pages Conservation and Stability · intermediate axis, Δt = 0.02, t_f = 80

Three limits we can name, and measure

The saddle amplifies errors too

Any error along the unstable direction grows like e^{λt}, just as ε does. Euler's growth rate ln(1 + λΔt)/Δt is lower than λ: with ε = 10^{-12} and Δt = 0.1 its first flip comes 1.4 s late.

Singular implicit steps

Crank-Nicolson's R(z) has a pole at z = 2; implicit Euler at z = 1. There, I − Δt J cannot be inverted and the step fails.

Floating point

Richardson differences stop falling near 10^{-12} (RK4, N = 5120): below that, rounding dominates truncation.

Source: flip-time runs (ε = 10^{-12}, Δt = 0.1) · Estabilidad.py · convergence table, page Precision

3 · Software, GUI and AI · Ignacio Pérez

Phase map · architecture · GUI · demo · how we worked with AI



Ten phases, one map: click any phase to jump to its evidence

01
Physical problem
24 Sep · slide 2

02
Analytical model
6 Oct · slide 4

03
Numerical formulation
28 Sep – 8 Oct · slide 10

04
Software architecture
6 Oct · slide 19

05
Numerical implementation
24 – 25 Oct · slide 13

06
GUI development
25 Oct – 2 Nov · slide 21

07
Integration
27 Oct · slide 20

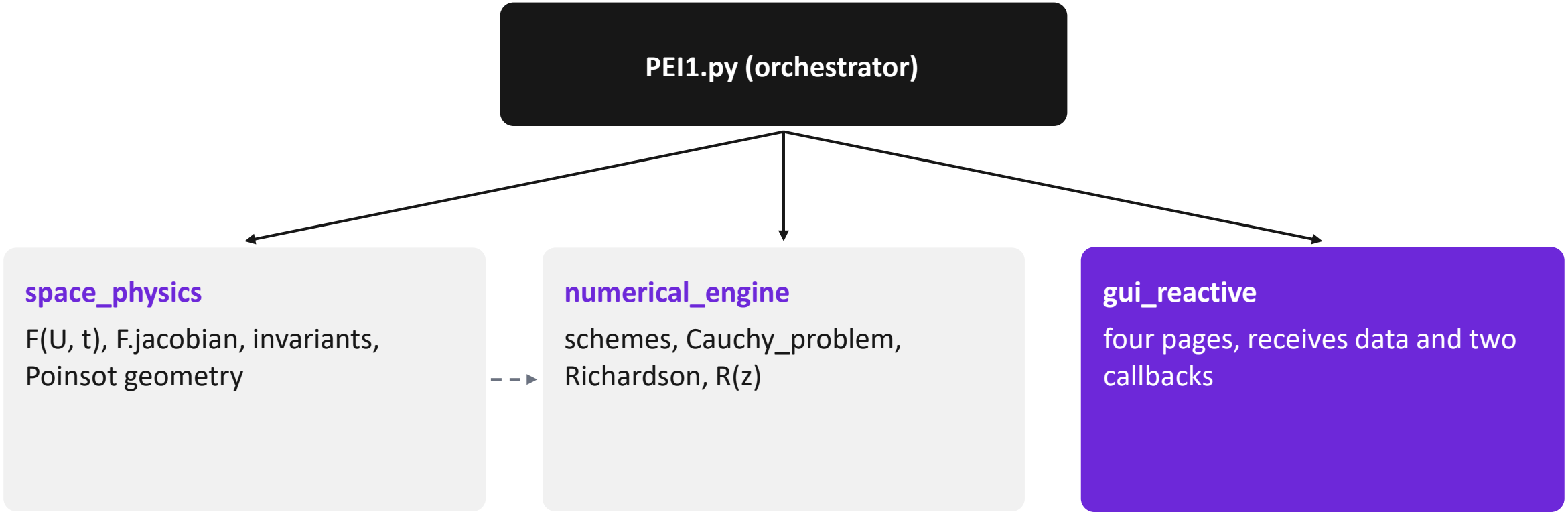
08
Testing and validation
28 Oct · slide 15

09
Debugging and iteration
24 Oct – 2 Nov · slide 25

10
Final application
2 Nov · slide 22

Click a phase to jump to its slide; the eyebrow of every software slide links back here.

Three layers, one orchestrator: the GUI imports neither physics nor engine



one leak: physics imports partition()

The GUI imports neither layer: it receives data and two callbacks.

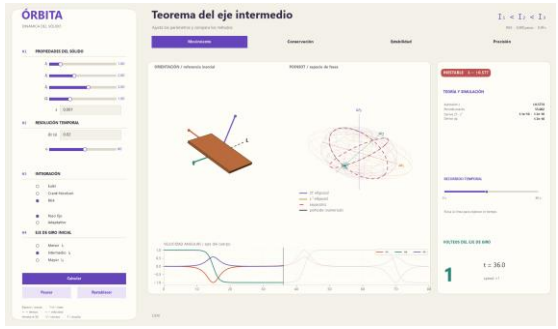
Source: imports of active/ and PEI1/PEI1.py

One click on Calcular follows one path through named functions

01 leer_parametros	GUI reads sliders and selectors: numbers and a method name, never a function
02 al_calcular	callback into PEI1.py: picks the scheme, builds F and U_0
03 integrar_trayectoria	Cauchy_problem with protection: partial result and a warning if it stops
04 preparar_pagina	invariants, regions, Richardson computed only when a page needs them
05 dibujar	each page draws data; the timer only moves the current instant

Source: active/gui_reactive/README.md · PEI1/PEI1.py

Four pages, each answering one question of the statement









Motion

What does the body do? Box, Poinsot surfaces, separatrix, ω(t)

Conservation

How much do 2T, L^2 and |q| drift with each scheme?

Stability

Where do the modes $z = \lambda\Delta t$ fall relative to $|R(z)| \le 1$?

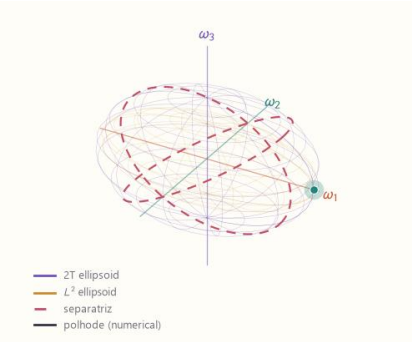
Precision

What order does each scheme reach, measured by Richardson?

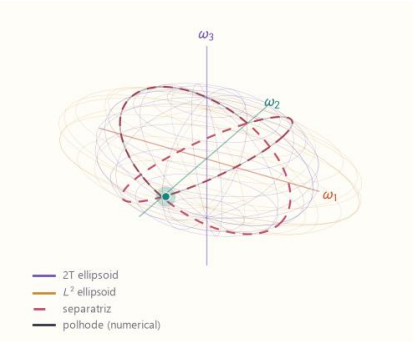
Screenshots captured by presentacion/figuras/capturar_gui.py from the real program

Live demo: three axes, then the cases that break things

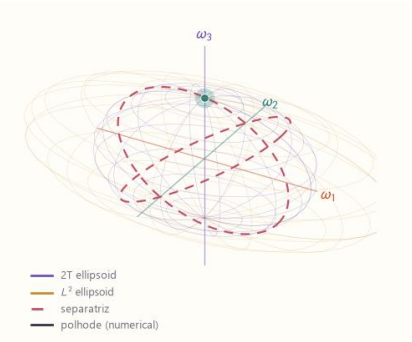
Run	Settings	What to show
Intermediate axis, RK4	**$I = (1, 2, 3)$, $\Omega = 1$, $\varepsilon = 10^{-3}$, $\Delta t = 0.02$**	**flips every 27.5 s, $\lambda = 0.577$**
Minimum and maximum axes	same, axis I_1 then I_3	closed polhodes, no flip
Euler on axis 2	Conservation page	2T grows 7 % in 80 s
PENDING: unusual cases	*$I_1 \approx I_2$; $I_1 + I_2 < I_3$; Δt beyond 2.83 s; adaptive step*	*to choose and rehearse*



Axis I_1: closed polhode

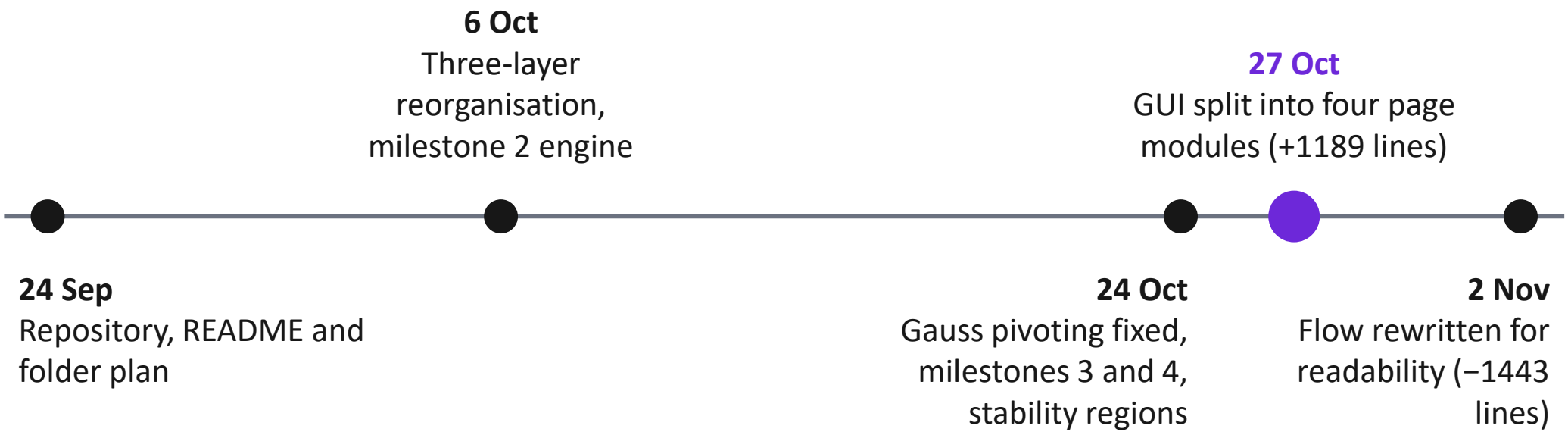


Axis I_2: separatrix



Axis I_3: closed polhode

Fifty-one commits, three authors, and two rewrites of the interface



24 Sep → 2 Nov 2026 · dots placed to scale in days

Source: git log, 51 commits (MP-19-creator 20, Rubennezc 17, Ignacio Pérez 14)

The AI worked inside rules we wrote before the code

In place

README per layer written as a contract before the engine and GUI code (6–8 Oct).

Fixed interfaces F(U, t) and method(U, Δt, t, F).

A regression script that compared every scheme bit for bit before and after the GUI refactor (28 Oct).

Screenshots of every page saved for review by a person.

Proposed

Tests that run on every commit instead of one-off scripts.

An instructions file for the assistant inside the repository.

Automatic architecture map generated from the imports.

Source: active/*/README.md · tmp/visual_review.py · tmp/visual-review/

Where the AI got it wrong, and how we noticed

A patch that hid a bug

temporal_schemes.Crank_Nicolson calls Newton with three arguments; Newton takes two. Instead of fixing it, PEI1.py checks Newton's signature and swaps in its own step.

A silent wrong answer

The fixed-point implicit step returned the Euler predictor for small Δt. Caught by comparing outputs: Crank-Nicolson equal to Euler, digit for digit.

Stale scaffolding

Both GUI test scripts in tmp/ still call functions the last rewrite removed; a physics docstring claims no imports from the engine.

PENDING: our episodes

Team to add two or three real prompt exchanges and the correction each needed.

Source: temporal_schemes.py · PEI1/PEI1.py (signature check) · numerical_engine/README.md · tmp/

The wing nut flips because its intermediate axis is a saddle. We can show it on the screen, measure where each scheme stops telling the truth, and point to the line of code that does it.

Individual defence · questions

Backup for questions

Finite-difference Jacobian · adaptive step · defects · knowledge map



The finite-difference Jacobian is exact for our F, up to rounding

Step h	max relative error vs analytic J (7 × 7, 200 random states)
0.1	$6 \cdot 10^{-15}$
10^{-3} (current)	**$8 \cdot 10^{-13}$**
10^{-6}	$6 \cdot 10^{-10}$

$E(h) \approx h^2/6 \cdot |F'''| + \varepsilon_{mach} \cdot |F| / h$. F is bilinear, so $F''' = 0$: only rounding is left, and a larger h is more accurate.

Source: NewtonRaphson.Jacobian(), central differences · check over 200 random states

Adaptive step: Richardson on a single step

01 Two estimates

U_1 = one step of Δt; U_2 = two steps of $\Delta t/2$

→

02 Error

$e = |U_2 - U_1|$

(2^q − 1)

→

03 Accept

if $e \le tol\,(1 + |U_2|)$, keep U_2

→

04 Next step

$\Delta t \leftarrow \Delta t \cdot 0.9$ (tol/e)^{1/(q+1)}, limited to [0.2, 2] Δt

Source: Cauchy_problem_adaptive() in Cauchy_Problem.py

Known defects from our own audit

Defect	Where	Priority
Crank_Nicolson calls Newton with 3 arguments	**temporal_schemes.py**	**high**
Newton raises RuntimeError, not caught by Cauchy_problem	NewtonRaphson.py	high
Gauss calls exit() on a singular matrix	NewtonRaphson.py	high
Leap_Frog keeps global state without start()	temporal_schemes.py	medium
Physics imports partition from the engine; root README out of date	eje_intermedio.py, README.md	low

Source: audit of the repository at commit f3eeb4b

PENDING: project knowledge map

To be generated from the repository: module imports, public functions and README sections, rendered as a graph. Proposal and steps in presentacion/backlog.md, task D.